



LAZY EVALUATION

Why Spark Builds a Plan Instead of Running Your Code

Transformations

Actions

DAG

Catalyst

Interview Ready in 6 Slides

Swipe to start learning →

L LAZY VS EAGER — THE CORE RULE

TRANSFORMATIONS (LAZY — BUILD THE PLAN, NO EXECUTION)

`.filter()`

`.select()`

`.groupBy()`

`.withColumn()`

`.join()`

ACTIONS (EAGER — TRIGGER FULL EXECUTION)

`.show()`

`.collect()`

`.count()`

`.write()`

The Golden Rule

A transformation only **records intent** — it returns a new DataFrame description, not a result. Spark doesn't touch a single row of data until an **Action** is called.

Interview Tip: If your code never calls an Action, Spark runs — literally nothing. No Job is ever submitted, no matter how many transformations you chain.

B WHAT HAPPENS BEHIND EACH TRANSFORMATION

Every chained transformation just extends a tree data structure sitting in the **Driver's memory** — nothing runs on the Executors yet.

The Plan Grows, One Call at a Time

1 **df.filter(...)** — adds a node to the Unresolved Logical Plan



2 **df.select(...)** — another node appended, still unresolved



3 **df.groupBy(...)** — plan keeps growing, zero data read so far



4 No JVM task exists yet — the Executors don't even know this chain exists

Interview Tip: The DataFrame you hold after a transformation is just a *pointer to a plan*, not a pointer to data.

C THE OPTIMIZATION PAYOFF

Because Spark sees the **entire chain** before running anything, Catalyst can rewrite it into something far cheaper than line-by-line eager execution.

What Laziness Buys You

1 **Predicate Pushdown** — filters move closer to the data source



2 **Projection Pruning** — unused columns dropped before reading



3 **Join Reordering** — Catalyst picks the cheapest join order



4 **Stage Coalescing** — adjacent narrow transforms fused into one stage

Interview Tip: This is why DataFrame code often beats hand-written RDD code — the optimizer only gets to act because execution was delayed.

D CODE TIMELINE

Watch **when** the cost actually lands — it's never where you'd expect.

Step 1 — Chain the Transformations

```
df2 = df.filter(col("age") > 30) \
        .select("name", "age") \
        .withColumn("bracket", ...)
```

⌚ 0 ms elapsed — nothing runs, only the plan is built

Step 2 — Call an Action

```
df2.show()
```

⚡ Action called → Job submitted → DAG built → Stages & Tasks execute NOW

Interview Tip: Timing your code around transformation lines is meaningless — all the real cost lands on the single line holding the Action.

Save this slide before your next Data Engineering interview

CONCEPT	TYPE	KEY INTERVIEW FACT
Transformation	Lazy	Builds plan only, returns new DataFrame
Action	Eager	Triggers Job submission
Logical Plan	Data structure	Grows with each transformation
Catalyst	Optimizer	Rewrites plan before physical execution
Job	Runtime unit	Created only when an Action runs

Follow for daily Data Engineering content

@chintapradeep

Spark · Python · SQL · Interview Prep